



**Wydział Matematyki
i Nauk Informatycznych**

POLITECHNIKA WARSZAWSKA

Algorytmy zaawansowane

Komiwojażer

Porównanie dwóch algorytmów aproksymacyjnych
znajdujących rozwiązanie problemu Komiwojażera

Dokumentacja wstępna

Zespół:

inż. Marcin Cieszyński

inż. Radosław Głasek

inż. Adam Grącikowski

Prowadzący:

dr Michał Tuczyński

Oświadczamy, że niniejsza praca, stanowiąca podstawę do uznania osiągnięcia efektów uczenia się
z przedmiotu *Algorytmy zaawansowane*, została wykonana samodzielnie.

Warszawa, 24 maja 2026

Spis treści

1	Wstęp	2
1.1	Cel projektu	2
1.2	Przedstawienie problemu	2
2	Opis rozwiązań problemu	3
2.1	Algorytm 2-aproksymacyjny	3
2.1.1	Analiza jakości aproksymacji	3
2.1.2	Analiza złożoności czasowej	3
2.2	Algorytm Christofidesa	3
2.2.1	Analiza jakości aproksymacji	4
2.2.2	Analiza złożoności obliczeniowej	5
2.2.3	Cykl Eulera	6
2.2.4	Algorytm Minimalnego Skojarzenia Doskonałego	6
3	Specyfikacja wejścia i wyjścia	10
3.1	Format danych wejściowych	10
3.2	Format danych wyjściowych	10
3.3	Przykłady	10
4	Generowanie instancji testowych	12
4.1	Składnia poleceń	12
4.1.1	Dostępne argumenty i flagi	12
4.2	Przykłady użycia	12
5	Instrukcja uruchomienia programu	13
5.1	Składnia poleceń	13
5.2	Dostępne argumenty i flagi	13
5.3	Przykłady użycia	13

1 Wstęp

Problem komiwojażera (ang. *Traveling Salesperson Problem*, TSP) to jeden z klasycznych problemów optymalizacji kombinatorycznej, należący do klasy problemów NP-trudnych.

1.1 Cel projektu

Przedmiotem projektu jest implementacja oraz szczegółowe porównanie dwóch algorytmów aproksymacyjnych rozwiązujących metryczną wersję problemu komiwojażera. Porównanie ma na celu ewaluację obu podejść pod kątem jakości zwracanych rozwiązań w stosunku do czasu ich wykonania na instancjach testowych o zróżnicowanej wielkości.

1.2 Przedstawienie problemu

Zadanie komiwojażera polega na znalezieniu najkrótszej możliwej trasy, która zaczyna się w wybranym punkcie, odwiedza dokładnie jeden raz każdy z pozostałych punktów, a następnie wraca do miejsca startu.

W ujęciu grafowym problem ten modeluje się za pomocą pełnego grafu nieskierowanego ważonego $G = (V, E)$, gdzie V to zbiór wierzchołków reprezentujących lokalizacje, a E to zbiór krawędzi łączących każdą parę wierzchołków. Każdej krawędzi przypisana jest waga za pomocą funkcji kosztu $c : E \rightarrow \mathbb{N}$. Zastosowanie dziedziny liczb naturalnych oznacza, że operujemy na całkowitoliczbowych, nieujemnych wartościach odległości. Naszym celem jest znalezienie cyklu Hamiltona C o minimalnym sumarycznym koszcie krawędzi.

Wersja metryczna problemu, którą rozpatrujemy w tym projekcie, nakłada na funkcję kosztu dodatkowe ograniczenia, upodabniające graf do rzeczywistej przestrzeni fizycznej. Najważniejszym z nich jest **nierówność trójkąta**. Oznacza ona, że bezpośrednia podróż z wierzchołka u do w nigdy nie jest droższa niż podróż przez jakikolwiek wierzchołek pośredni v . Formalnie dla każdych trzech wierzchołków $u, v, w \in V$ zachodzi:

$$c(u, w) \leq c(u, v) + c(v, w)$$

Dodatkowo zakładamy symetrię, czyli $c(u, v) = c(v, u)$.

2 Opis rozwiązań problemu

Do porównania wybrano dwa algorytmy: 2-aproksymacyjny[2] oraz Christofidesa[1].

2.1 Algorytm 2-aproksymacyjny

Algorytm 2-aproksymacyjny opiera się na znalezieniu minimalnego drzewa rozpinającego (MST), i za jego pomocą wyznaczenie cyklu Hamiltona. Niech $G = (V, E)$ będzie grafem pełnym o nieujemnych wagach krawędzi, a funkcja kosztu c spełnia nierówność trójkąta.

Algorithm 1 Algorytm 2-aproksymacyjny(G, c)

- 1: $r \leftarrow$ wybierz dowolny wierzchołek z $G.V$
 - 2: $T \leftarrow \text{MST}(G, c, r)$ $\triangleright$ minimalne drzewo rozpinające
 - 3: $L \leftarrow$ lista wierzchołków w porządku preorder drzewa T
 - 4: **return** cykl H , który odwiedza wierzchołki zgodnie z listą L
-

2.1.1 Analiza jakości aproksymacji

Niech H^* będzie optymalnym cyklem Hamiltona w grafie G . Poprzez usunięcie jednej krawędzi z H^* , otrzymamy drzewo rozpinające, zatem koszt minimalnego drzewa rozpinającego T stanowi ograniczenie dolne kosztu rozwiązania optymalnego:

$$c(T) \leq c(H^*) \quad (1)$$

Definiujemy pełny spacer W jako ciąg wierzchołków powstały przez obejście drzewa T „dookoła” (odwiedzając krawędzie w obu kierunkach). W takim spacerze każda krawędź drzewa T występuje dokładnie dwukrotnie, zatem:

$$c(W) = 2c(T) \quad (2)$$

Z nierówności 1 wynika, że taki spacer jest maksymalnie 2 razy dłuższy niż rozwiązanie optymalne:

$$c(W) \leq 2c(H^*) \quad (3)$$

Spacer W nie jest cyklem Hamiltona, natomiast korzystając z tego, że G grafem pełnym, możemy stworzyć cykl H odwiedzając każdy wierzchołek (oprócz początkowego) tylko raz - odwiedzamy wierzchołki w kolejności ich pierwszego pojawienia się w spacerze W . Wiemy, że każde takie skrócenie spaceru nie może nam wydłużyć jego długości, ponieważ c spełnia nierówność trójkąta:

$$c(H) \leq c(W) \quad (4)$$

W ostateczności otrzymujemy cykl o koszcie:

$$c(H) \leq 2c(H^*) \quad (5)$$

Co oznacza, że tak wyznaczony cykl H będzie nie dłuższy niż dwukrotność długości rozwiązania optymalnego.

2.1.2 Analiza złożoności czasowej

Do wyznaczenia minimalnego drzewa rozpinającego skorzystano z algorytmu Prima. Złożoność tego algorytmu bez zastosowania kopców Fibonacciego to: $O(|E| \log |V|)$. Wyznaczenie porządku wstępnego (ang. *preorder*) w drzewie to: $O(|V|)$. W grafie pełnym $|E| = O(|V|^2)$, a więc łączna złożoność czasowa algorytmu wynosi:

$$O(|V|^2 \log |V|) \quad (6)$$

2.2 Algorytm Christofidesa

Drugim z zastosowanych algorytmów jest algorytm przedstawiony przez Christofidesa [1] (alg. 2). Umożliwia on znalezienie cyklu Hamiltona o koszcie nie większym niż $\frac{3}{2}\text{OPT}$.

Algorithm 2 Algorytm Christofidesa

```
1:  $T^* \leftarrow$  minimalne drzewo rozpinające grafu  $G$ 
2:  $O \leftarrow \emptyset$  ▷ zbiór wierzchołków o nieparzystym stopniu
3: for każdy wierzchołek  $v \in V$  do
4:   if  $d_v(T^*)$  jest nieparzyste then
5:      $O \leftarrow O \cup \{v\}$ 
6:   end if
7: end for
8:  $M_O^* \leftarrow$  skojarzenie doskonałe o najmniejszym koszcie w podgrafie  $\langle O \rangle$ 
9:  $G^e \leftarrow (V, E_{T^*} \cup E_{M_O^*})$  ▷ spójny multigraf o wszystkich stopniach parzystych
10:  $\Phi^e \leftarrow$  cykl Eulera w  $G^e$ 
11:  $V \leftarrow \emptyset$  ▷ zbiór wierzchołków już odwiedzonych
12:  $\Phi_H \leftarrow$  pusta ścieżka
13: for każdy wierzchołek  $v$  w kolejności wyznaczonej przez  $\Phi^e$  do
14:   if  $v \notin V$  then
15:     dołącz  $v$  do  $\Phi_H$ 
16:      $V \leftarrow V \cup \{v\}$ 
17:   end if
18: end for
19: zamknij  $\Phi_H$  krawędzią z ostatniego do pierwszego wierzchołka
```

2.2.1 Analiza jakości aproksymacji

Przypomnijmy najpierw lemat o uściskach dłoni:

Lemat 2.1. *Suma stopni wierzchołków grafu jest równa dwukrotności liczby jego krawędzi.*

$$\sum_{v \in V} \deg(v) = 2|E|$$

Rozważmy problem komiwojażera dla grafu pełnego $G = (V, E)$, $|V| = n$. Niech macierz kosztów $[c_{ij}]$ spełnia nierówność trójkąta. Niech $T^* = (V, E_{T^*})$ będzie drzewem rozpinającym grafu G o najmniejszym koszcie $c(T^*)$. Niech $V^O(T^*)$ będzie zbiorem wierzchołków G o nieparzystym stopniu w drzewie T^* :

$$V^O(T^*) = \{v_i : 2 \nmid d_i(T^*)\}$$

($d_i(T^*)$ jest stopniem wierzchołka $v_i \in V$ w T^*). Z lematu o uściskach dłoni 2.1 (zastosowanego do T^*) liczba elementów tego zbioru: $|V^O(T^*)|$ jest zawsze parzysta.

Rozważmy teraz podgraf $\langle V^O(T^*) \rangle$ indukowany przez zbiór $V^O(T^*)$ wierzchołków. Z parzystej mocy zbioru $V^O(T^*)$ wynika, że zawsze istnieje skojarzenie doskonałe w grafie $V^O(T^*)$, tj. takie skojarzenie, które zawsze zawiera $1/2|V^O(T^*)|$ krawędzi. Niech $M_O^* = (V^O(T^*), E_{M_O^*})$ będzie skojarzeniem doskonałym $\langle V^O(T^*) \rangle$ o najmniejszym koszcie $c(M_O^*)$.

Twierdzenie 2.2. *Cykl Hamiltona Φ_H grafu G może być znaleziony z kosztem $c(\Phi_H) \leq c(T^*) + c(M_O^*) < \frac{3}{2}c(\Phi^*)$ gdzie $c(\Phi^*)$ jest kosztem optymalnego cyklu Hamiltona Φ^* .*

Lemat 2.3. *Dla problemu komiwojażera grafu G o n wierzchołkach i parzystym n zachodzi $c(M^*) \leq \frac{1}{2}c(\Phi^*)$, gdzie M^* jest skojarzeniem doskonałym grafu G o najmniejszym koszcie i Φ^* jest optymalnym cyklem Hamiltona.*

Dowód. Rozważmy $\Phi^* = (v_{i_1}, v_{i_2}, \dots, v_{i_n})$. Zaczynając z wierzchołka v_{i_1} i przechodząc przez cykl Φ^* , przyporządkowujemy kolejne krawędzie do dwóch zbiorów M_1 i M_2 naprzemiennie. Przykładowo zaczynając od M_1 otrzymamy:

$$M_1 = \{(v_{i_1}, v_{i_2}), (v_{i_3}, v_{i_4}), \dots, (v_{i_{n-1}}, v_{i_n})\}$$

oraz

$$M_2 = \{(v_{i_2}, v_{i_3}), (v_{i_4}, v_{i_5}), \dots, (v_{i_n}, v_{i_1})\}$$

M_1 i M_2 są skojarzeniami G oraz zachodzi

$$c(M_1) + c(M_2) = c(\Phi^*)$$

M_1 oraz M_2 zostały wybrane w sposób dowolny, możemy więc założyć $c(M_1) \leq c(M_2)$ bez utraty ogólności. Wówczas:

$$c(M^*) \leq c(M_1) \leq \frac{1}{2}c(\Phi^*)$$

co kończy dowód lematu. $\square$

Pokażemy teraz dowód twierdzenia 2.2.

Dowód. Z definicji T^* dla grafu G zachodzi

$$c(T^*) \leq c(\Phi_p^*) < c(\Phi^*) \quad (7)$$

gdzie $c(\Phi_p^*)$ jest minimalnym kosztem ścieżki Hamiltona grafu G , a Φ_p^* jest ścieżką Hamiltona o minimalnym koszcie (ostatnia nierówność przechodzi w $\leq$ przy dopuszczeniu krawędzi o koszcie 0).

Graf $G^e = (V, E_{T^*} \cup E_{M_O^*})$ jest eulerowski tj. ma wszystkie wierzchołki parzystego stopnia, ponieważ M_O^* jest skojarzeniem wszystkich wierzchołków nieparzystego stopnia w T^* . Więc G^e zawiera cykl Eulera $\Phi^e = (v_{i_1}, v_{i_2}, \dots, v_{i_k})$, który przechodzi przez wszystkie wierzchołki $v_i^* \in V$ co najmniej raz. Niech $c(\Phi^e)$ będzie kosztem Φ^e tj.

$$c(\Phi^e) = c(T^*) + c(M_O^*) \quad (8)$$

Jeśli Φ_O^* jest rozwiązaniem problemu komiwojażera dla problemu zdefiniowanego przez indukowany podgraf $\langle V^O(T^*) \rangle$ to z lematu 2.3 mamy $c(M_O^*) \leq \frac{1}{2}c(\Phi_O^*)$. Ponadto $c(\Phi_O^*) \leq c(\Phi^*)$ otrzymujemy więc

$$c(M_O^*) \leq \frac{1}{2}c(\Phi^*) \quad (9)$$

z 7, 8 oraz 9 otrzymujemy

$$c(\Phi^e) < \frac{3}{2}c(\Phi^*)$$

Rozważmy przejście Φ^e zaczynając z v_{i_1} do momentu, gdy wierzchołek v_{i_r} (wcześniej odwiedzony) zostaje osiągnięty tj. $v_{i_r} = [v_{i_1}, \dots, v_{i_{r-1}}]$. Niech v_{i_s} będzie pierwszym wierzchołkiem następującym po wierzchołku v_{i_r} w Φ^e , który nie został jeszcze odwiedzony. Rozważmy cykl $\Phi_1 = (v_{i_1}, \dots, v_{i_{r-1}}, v_{i_s}, \dots, v_{i_k})$ otrzymany z Φ^e przez zastąpienie ścieżki $P_{rs} = (v_{i_{r-1}}, v_{i_r}, \dots, v_{i_{s-1}}, v_{i_s})$ w tym cyklu krawędzią $(v_{i_{r-1}}, v_{i_s})$. Z nierówności trójkąta mamy:

$$c_{i_{r-1}i_s} \leq \sum_{(v_i, v_j) \in P_{rs}} c_{ij}$$

gdzie P_{rs} jest także używany jako zbiór krawędzi ścieżki P_{rs} . Więc koszt skróconego cyklu jest nie większy: $c(\Phi^e) \geq c(\Phi_1)$.

W podobny sposób, zaczynając z przejścia Φ_1 , można utworzyć cykl Φ_2 ze ścieżką z Φ_1 zastąpioną przez krawędź i $c(\Phi_1) \geq c(\Phi_2)$. W końcu otrzymamy cykl Hamiltona grafu G :

$$c(\Phi_H) \leq \dots \leq c(\Phi_1) \leq c(\Phi^e) < \frac{3}{2}c(\Phi^*)$$

$\square$

2.2.2 Analiza złożoności obliczeniowej

Algorytm składa się z trzech części:

- konstrukcji minimalnego drzewa rozpinającego – o koszcie $O(n^2 \log(n))$ (alg. Prima),
- znalezienia skojarzenia doskonałego o najmniejszym koszcie – o koszcie $O(n^2 m)$. W przypadku problemu komiwojażera ($m = O(n^2)$) daje to złożoność $O(n^4)$ (możliwe jest osiągnięcie teoretycznej złożoności $O(n^3)$, ale zaimplementowany i opisany wariant pierwotno-dualny cechuje się wyższą złożonością),
- znalezienia cyklu Eulera oraz jego modyfikacji – koszt liniowy ze względu na liczbę krawędzi, zatem w szczególności o złożoności $O(n^2)$.

Zatem koszt obliczeniowy algorytmu jest równy

$$O(n^2 \log(n)) + O(n^4) + O(n^2) = O(n^4)$$

2.2.3 Cykl Eulera

Do wyznaczenia cyklu Eulera wykorzystamy algorytm Hierholzera (alg. 3), który pozwala na znalezienie ścieżki w czasie liniowym względem liczby krawędzi.

Algorithm 3 Algorytm Hierholzera (Wyznaczanie cyklu Eulera)

```

1:  $S \leftarrow$  pusty stos,
2:  $P \leftarrow$  pusta lista ▷ Konstruowany cykl Eulera
3: Wybierz dowolny wierzchołek startowy  $v \in V$ 
4:  $S.push(v)$ 
5: while  $S$  nie jest pusty do
6:    $u \leftarrow S.top()$ 
7:   if  $u$  posiada nieodwiedzone krawędzie incydentne then
8:     Wybierz krawędź  $(u, w)$ , usuń ją z grafu  $G^e$ 
9:      $S.push(w)$ 
10:  else
11:     $P.add(S.pop())$ 
12:  end if
13: end while
14: return  $P$  (wierzchołki cyklu  $\Phi^e$  w odwrotnej kolejności)

```

Algorytm Hierholzera ma złożonością czasową $O(m^*)$, gdzie m^* jest sumaryczną liczbą krawędzi w drzewie T^* oraz skojarzeniu M_O^* . W szczególności $O(m^*) = O(n^2)$.

2.2.4 Algorytm Minimalnego Skojarzenia Doskonałego

W algorytmie Christofidesa (alg. 2) kluczowe jest wyznaczenie minimalnego skojarzenia doskonałego M_O^* (*MWPM* ang. *Minimum Weight Perfect Matching*). Jest to realizowane przy użyciu algorytmu Edmonds'a (ang. *Blossom Algorithm*) w wariacie pierwotno-dualnym. Opis bazuje na rozwiązaniu Dilsona Lucasa Pereiry [4], które implementuje algorytm przedstawiony przez Gerardsa [3].

Zrozumienie działania algorytmu wymaga spojrzenia na problem przez pryzmat programowania liniowego. Problem znalezienia skojarzenia doskonałego o minimalnym koszcie można sformułować wprowadzając zmienną decyzyjną $x_e \in \{0, 1\}$, która określa, czy krawędź $e \in E$ należy do skojarzenia. Relaksacja liniowa problemu pierwotnego (ang. *primal*) przyjmuje postać minimalizacji kosztu:

$$\min \sum_{e \in E} c_e x_e \quad (10)$$

przy następujących ograniczeniach:

$$\begin{cases} \sum_{e \in \delta(v)} x_e = 1 & \forall v \in V & (\text{każdy wierzchołek ma dokładnie jedną parę}) \\ \sum_{e \in \delta(B)} x_e \geq 1 & \forall B \in \mathcal{O} & (\text{każdy kwiat musi mieć krawędź wychodzącą}) \\ x_e \geq 0 & \forall e \in E \end{cases}$$

gdzie $\mathcal{O}$ to rodzina wszystkich podzbiorów wierzchołków o nieparzystej mocy (potencjalnych kwiatów), a $\delta(S)$ to zbiór krawędzi z dokładnie jednym końcem w zbiorze S .

Zgodnie z zasadami dualności programowania liniowego, każdemu ograniczeniu z problemu pierwotnego odpowiada zmienna w problemie dualnym (równościom odpowiadają zmienne swobodne, a nierównościom zmienne ograniczone). Przypisujemy potencjał (zmienną) $y_v \in \mathbb{R}$ równościom oraz nieujemny potencjał $z_B \geq 0$ nierównościom dla kwiatów. Problem dualny polega na maksymalizacji sumy tych potencjałów:

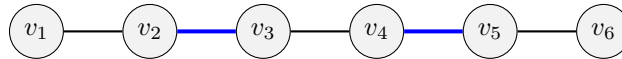
$$\max \left(\sum_{v \in V} y_v + \sum_{B \in \mathcal{O}} z_B \right) \quad (11)$$

przy ograniczeniu dla każdej krawędzi $e = (u, v) \in E$:

$$y_u + y_v + \sum_{B: e \in \delta(B)} z_B \leq c_e \quad (12)$$

Z twierdzenia o równowadze komplementarnej (ang. *complementary slackness*), dla optymalnych rozwiązań problemu pierwotnego i dualnego iloczyn wartości zmiennej decyzyjnej i odpowiadającego jej luzu z drugiego problemu musi wynosić zero. Ponieważ interesują nas krawędzie faktycznie wchodzące w skład skojarzenia (dla których zmienna pierwotna $x_e = 1 > 0$), wymusza to, aby ich dualny luz wynosił dokładnie zero. Struktury i pojęcia używane w algorytmie przedstawiono poniżej:

- **Las alternujący (węgierski):** Las alternujący jest strukturą pomocniczą budowana przez algorytm w celu znalezienia ścieżki powiększającej (która pozwala zwiększyć rozmiar skojarzenia). Las składa się z drzew ukorzenionych w wierzchołkach, które nie mają jeszcze przypisanej pary. Ścieżki w takich drzewach składają się naprzemiennie z krawędzi nienależących i należących do aktualnego skojarzenia.

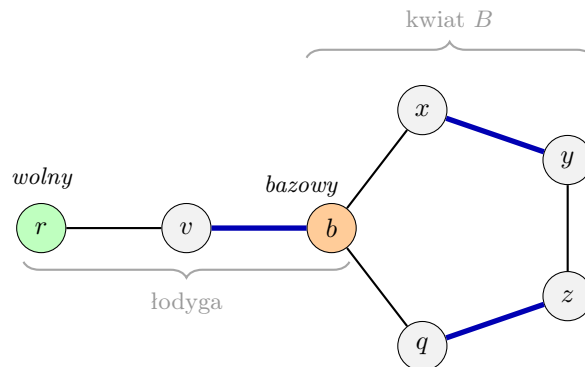


Rysunek 1: Przykład ścieżki powiększającej. Pogrubione (niebieskie) linie to krawędzie należące do aktualnego skojarzenia M , natomiast cieńsze (czarne) do niego nie należą. Skrajne wierzchołki v_1 i v_6 są wolne (nieskojarzone). Zamiana ról krawędzi na ścieżce zwiększy rozmiar skojarzenia o jeden.

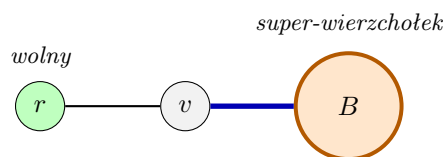
- **Kwiat (ang. *blossom*):** Kwiatem B nazywamy cykl nieparzystej długości $2k + 1$ ($k \geq 1$) taki, że dokładnie k spośród jego krawędzi należy do aktualnego skojarzenia M , a jeden wierzchołek cyklu, nazywany wierzchołkiem bazowym (ang. *base, tip*), jest nieskojarzony wewnątrz B (tj. obie krawędzie cyklu incydentne z nim nie należą do M).

Łodygą (ang. *stem*) kwiatu B nazywamy naprzemienną ścieżkę P łączącą wierzchołek bazowy b kwiatu z pewnym wolnym (nieskojarzonym) wierzchołkiem r grafu, w której krawędzie na przemian nie należą i należą do M (zaczynając od krawędzi należącej do M wychodzącej z b).

Kwiaty zaburzają budowę lasu alternującego i uniemożliwiają proste znalezienie ścieżki powiększającej. Algorytm rozwiązuje ten problem poprzez ich identyfikację i tymczasowe kontraktowanie do pojedynczych super-wierzchołków.



(a) Graf przed kontrakcją. Kwiat $B = \{b, x, y, z, q\}$ to cykl długości 5 z $k=2$ krawędziami skojarzenia (niebieskie). Wierzchołek bazowy b (pomarańczowy) ma obie swoje krawędzie w cyklu poza skojarzeniem. Łodyga $r-v-b$ łączy kwiat z wolnym wierzchołkiem r (zielony); jej krawędź vb należy do skojarzenia.



(b) Graf po kontrakcji kwiatu B do pojedynczego super-wierzchołka. Łodyga pozostaje niezmienną.

— krawędź należąca do skojarzenia M
 — krawędź nienależąca do M

Rysunek 2: Kontrakcja kwiatu w algorytmie Edmondsa.

- **Dopuszczalność dualna i luzy** Przekształcając nierówność z problemu dualnego (12), dla każdej krawędzi $(v_i, v_j) \in E$ definiuje się jej luz (ang. *slack*):

$$slack(v_i, v_j) = c_{ij} - \left(y_i + y_j + \sum_{B:(v_i, v_j) \in \delta(B)} z_B \right) \geq 0 \quad (13)$$

Krawędź jest uznawana za dopuszczalną (ang. *tight*) do wykorzystania w algorytmie tylko wtedy, gdy jej luz wynosi dokładnie zero ($slack = 0$). W kontekście budowanego lasu alternującego oznacza to, że jeśli wierzchołek v_i należy już do pewnego drzewa w lesie (jako wierzchołek będący w parzystej odległości od korzenia), to dopuszczalna krawędź (v_i, v_j) pozwala rozszerzyć to konkretne drzewo o nowy wierzchołek v_j (oraz ewentualnie o jego partnera, z którym jest już skojarzony).

W przeciwieństwie do algorytmu dla grafów dwudzielnych, konieczność obsługi cykli o nieparzystej długości wymusza kontrakcję kwiatów. Prezentowana implementacja w każdej iteracji odbudowuje węgierski las naprzemienny, włączając w to detekcję cykli. Gdy budowa lasu utknie, algorytm oblicza minimalną możliwą zmianę potencjałów dualnych, co skutkuje wyzerowaniem luzu dla nowych krawędzi. Dzięki temu stają się one dopuszczalne, co pozwala na dalsze rozbudowywanie lasu alternującego.

Algorithm 4 Algorytm Edmonds’a (MWPM w ujęciu pierwotno-dualnym)

- 1: Zredukuj koszty: $c_e \leftarrow c_e - \min_{e'} c_{e'}$ dla wszystkich $e \in E$
 - 2: Inicjalizuj skojarzenie $M \leftarrow \emptyset$, luzy krawędzi (jako koszty) oraz potencjały dualne
 - 3: **while** M nie jest skojarzeniem doskonałym **do**
 - 4: Wykonaj szybką heurystykę skojarzenia wierzchołków bez pary
 - 5: Zbuduj las alternujący oparty na krawędziach o zerowym luzie (*Grow*) ▷ *Grow*
 za pomocą struktury BFS potrafi odnaleźć ścieżkę powiększającą i powiększyć M (*Augment*)
 lub odnaleźć cykl nieparzysty i skontraktować go (*Blossom*)
 - 6: Oblicz minimalną zmianę dualną (pełne skanowanie krawędzi z luzem)
 - 7: Aktualizuj potencjały dualne, luzy i ewentualnie odblokuj kwiaty
 - 8: Zresetuj strukturę lasu dla nowej iteracji
 - 9: **end while**
 - 10: **return** M
-

Poszczególne kroki algorytmu 4 przedstawiono poniżej.

Heurystyka początkowa Aby zminimalizować liczbę kosztownych iteracji głównej pętli algorytmu, nie rozpoczyna on od pustego skojarzenia. Budowane jest możliwie największe skojarzenie startowe z wykorzystaniem zachłannej heurystyki. Iteracyjnie sprawdzane są nieskojarzone wierzchołki i dobierani im są tacy wolni sąsiedzi, których stopień w grafie jest najmniejszy. Dzięki temu szybciej tworzone są początkowe pary, a struktury algorytmu muszą obsłużyć mniejszą pulę izolowanych wierzchołków.

Rozbudowa lasu alternującego (ang. *Grow*) Właściwa faza poszukiwania ścieżki powiększającej opiera się na przeszukiwaniu grafu wszerz. Korzeniami lasu alternującego są wierzchołki bez przypisanej pary. Przeszukiwanie wędruje wyłącznie po krawędziach *dopuszczalnych*, czyli takich, których luz dualny jest aktualnie równy zero. W trakcie ekspansji wierzchołkom w lesie nadawane są etykiety parzystości (odległości od korzenia drzewa). W tej fazie mogą wystąpić trzy scenariusze:

- **Rozszerzenie:** Jeśli algorytm trafi na wierzchołek wolny od etykiet, dołącza go do struktury lasu wraz z jego partnerem ze skojarzenia, nadając im odpowiednio status nieparzysty i parzysty.
- **Kwiat (ang. *Blossom*):** Jeśli dopuszczalna krawędź połączy dwa parzyste wierzchołki z tego samego drzewa, algorytm identyfikuje cykl nieparzysty (kwiat). Cykl ten jest natychmiast kontraktowany do pojedynczego super-wierzchołka o statusie parzystym, co pozwala pominąć problem zapętlenia na etapie wyznaczania ścieżki powiększającej.
- **Ścieżka powiększająca (ang. *Augment*):** Jeśli badana krawędź połączy dwa wierzchołki parzyste należące do dwóch oddzielnych drzew, oznacza to odnalezienie ścieżki powiększającej

łączącej dwa pierwotnie wolne wierzchołki. Algorytm odwraca przypisania krawędzi wzdłuż odnalezionej trasy, zwiększając całkowity rozmiar skojarzenia o 1. Następnie struktura lasu jest całkowicie resetowana.

Aktualizacja potencjałów dualnych Często zdarza się, że faza rozbudowy lasu utknie – brakuje krawędzi dopuszczalnych, które pozwalałyby na dalszą eksplorację grafu, a skojarzenie doskonałe wciąż nie zostało osiągnięte. Wówczas algorytm ucieka się do modyfikacji problemu dualnego. Głównym celem tej operacji jest przesunięcie potencjałów tak, aby co najmniej jedna nowa krawędź uzyskała $slack = 0$, przy jednoczesnym zachowaniu warunku dopuszczalności dualnej, tj. $slack \geq 0$ dla wszystkich krawędzi.

Aby to osiągnąć, wyznaczana jest maksymalna bezpieczna wartość zmiany, ozn. ϵ . Jest ona ograniczona przez trzy czynniki, zapobiegające powstaniu ujemnych luzów:

1. **Krawędzie między wierzchołkami parzystymi a nieodwiedzonymi:** Ponieważ potencjał wierzchołka parzystego wzrośnie o ϵ , luz na krawędzi łączącej go z wierzchołkiem spoza lasu zmaleje o ϵ . Zatem zmiana nie może przekroczyć aktualnego luzu tej krawędzi: $\epsilon \leq slack(e)$.
2. **Krawędzie między dwoma wierzchołkami parzystymi:** Zwiększenie potencjałów obu wierzchołków na końcach takiej krawędzi o ϵ spowoduje spadek luzu aż o 2ϵ . Wymusza to ograniczenie: $\epsilon \leq \frac{slack(e)}{2}$.
3. **Zablokowane kwiaty o statusie nieparzystym:** Zgodnie z modelem programowania liniowego, potencjał dualny kwiatu (zmienna z_B) nie może być ujemny ($z_B \geq 0$). Skoro potencjały węzłów nieparzystych będą zmniejszane, modyfikacja nie może przekroczyć ich obecnego potencjału: $\epsilon \leq z_B$.

Ostateczna wartość ϵ to absolutne minimum z powyższych ograniczeń. Po jej wyznaczeniu, algorytm aktualizuje cały układ:

- potencjały dualne wierzchołków (i skontraktowanych kwiatów) o statusie **parzystym** rosną o ϵ ,
- potencjały dualne węzłów o statusie **nieparzystym** maleją o ϵ ,
- potencjały wierzchołków **nieodwiedzonych** (poza lasem) pozostają bez zmian.

Konsekwencją modyfikacji potencjałów węzłów jest matematyczna zmiana wartości luzów na krawędziach. Przykładowo, krawędzie między węzłem parzystym a nieodwiedzonym zmniejszają swój luz o ϵ , a między dwoma parzystymi – o 2ϵ . Analogicznie luzy krawędzi incydentnych z węzłami nieparzystymi odpowiednio rosną, co chroni je przed naruszeniem warunku $slack \geq 0$.

Dzięki temu algorytm zawsze osiąga postęp: wprowadzana jest nowa krawędź dopuszczalna (co umożliwia dalszą rozbudowę lasu, skontraktowanie nowego kwiatu lub znalezienie ścieżki powiększającej) albo potencjał dualny zablokowanego kwiatu spada do zera ($z_B = 0$). W drugim przypadku kwiat traci swoje ograniczenie, zostaje odblokowany i następuje jego de-kontrakcja (rozbiecie z powrotem na cykl wierzchołków), co otwiera nowe ścieżki do eksploracji.

Złożoność obliczeniowa Pod względem teoretycznym algorytm Edmonds’a można zaimplementować w czasie $O(n^3)$ stosując złożone struktury danych. Implementacja, na której się opieramy, ma jednak inną specyfikę. Pętla główna (alg. 4) wykonuje się do momentu znalezienia skojarzenia doskonałego. Pomiedzy powiększeniami skojarzenia zachodzi maksymalnie $O(n)$ aktualizacji potencjałów dualnych (każda aktualizacja dualnych albo wprowadza nową krawędź dopuszczalną, albo odblokowuje kwiat, a tych jest $O(n)$), co w pesymistycznym wariancie wymusza powtórzenie głównej pętli $O(n^2)$ razy. Podczas każdej takiej iteracji wywoływane są pełne przeszukania struktury, co kosztuje $O(m)$ operacji. Całkowita złożoność tej konkretnej implementacji wynosi więc $O(n^2 \cdot m) = O(n^2 \cdot n^2) = O(n^4)$.

3 Specyfikacja wejścia i wyjścia

Program został zaimplementowany jako aplikacja konsolowa, która wczytuje instancję problemu z pliku tekstowego. Wynik działania algorytmu jest domyślnie wypisywany na standardowe wyjście (konsolę) w czytelnej dla użytkownika formie, a opcjonalnie może zostać zapisany do pliku tekstowego.

3.1 Format danych wejściowych

Plik wejściowy jest standardowym plikiem tekstowym (np. z rozszerzeniem `.txt`). Jego struktura opiera się na następujących zasadach:

- W pierwszej linii znajduje się pojedyncza liczba całkowita dodatnia N , określająca całkowitą liczbę wierzchołków w grafie.
- W kolejnych N liniach znajduje się macierz sąsiedztwa modelująca graf pełny. Każdy wiersz zawiera dokładnie N wartości oddzielonych białymi znakami (pojedynczą spacją lub tabulatorem).

Wartości w macierzy reprezentują wagi krawędzi (odległości) i są liczbami naturalnymi. Macierz jest symetryczna, posiada zera na głównej przekątnej, a zapisane w niej odległości spełniają nierówność trójkąta.

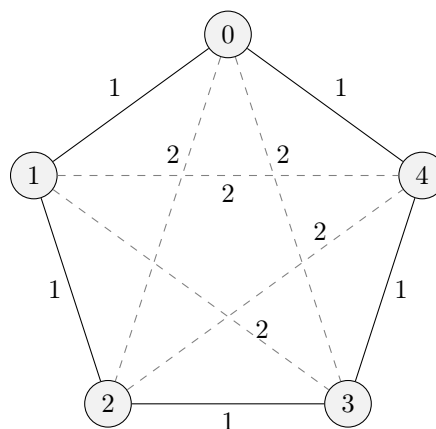
3.2 Format danych wyjściowych

Plik wyjściowy jest standardowym plikiem tekstowym (np. z rozszerzeniem `.txt`). Jego struktura opiera się na następujących zasadach:

- Pierwsza linia rozpoczyna się od frazy „Total Distance: ”, po której następuje całkowity koszt znalezionej rozwiązania (suma wag krawędzi tworzących cykl Hamiltona).
- Druga linia rozpoczyna się od frazy „Route: ”, po której następuje reprezentacja wyznaczonego cyklu Hamiltona. Identyfikatory wierzchołków wypisane są w kolejności odwiedzania i rozdzielone znakiem pauzy (-). Pierwszy i ostatni wierzchołek w tej linii są identyczne, domykając cykl.

3.3 Przykłady

Przeanalizujemy zawartość pliku wejściowego oraz wyjściowego na przykładzie grafu G przedstawionego na Rysunku 3.



Rysunek 3: Przykładowy graf wejściowy G .

Graf składa się z pięciu wierzchołków ($N = 5$). Krawędzie ułożone wzdłuż obwodu pięciokąta mają wagę 1, natomiast wszystkie „przekątne” (łącznie wierzchołki niebędące bezpośrednimi sąsiadami na obwodzie) mają wagę 2. Warto zauważyć, że wszystkie wagi krawędzi to liczby naturalne i spełniona jest tu nierówność trójkąta.

Zawartość pliku wejściowego dla grafu G przedstawia Rysunek 4.

```

5
0 1 2 2 1
1 0 1 2 2
2 1 0 1 2
2 2 1 0 1
1 2 2 1 0

```

Rysunek 4: Zawartość pliku wejściowego dla grafu G .

Wynik działania programu dla powyższej instancji został zilustrowany na Rysunku 5. Pierwsza linia pliku wyjściowego precyzuje całkowity koszt znalezionej trasy, wynoszący w tym przypadku 5. Jest to suma wag pięciu krawędzi wchodzących w skład wyznaczonego cyklu. Druga linia określa dokładną kolejność odwiedzania wierzchołków.

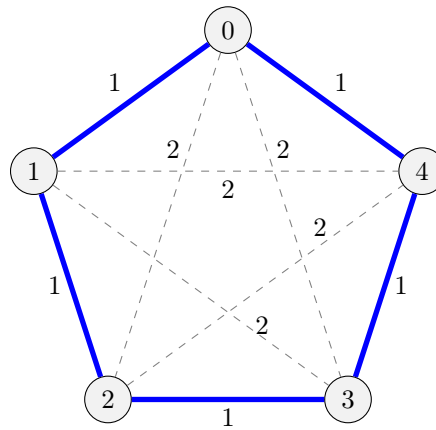
```

Total Distance: 5
Route: 0-1-2-3-4-0

```

Rysunek 5: Zawartość pliku wyjściowego dla grafu G .

W prezentowanym przykładzie wyznaczona ścieżka (0-1-2-3-4-0) przebiega wyłącznie po krawędziach o wadze 1 (Rysunek 6).



Rysunek 6: Rozwiązanie zapisane w pliku wyjściowym.

4 Generowanie instancji testowych

Aplikacja została zrealizowana w środowisku .NET 9 jako program konsolowy.

4.1 Składnia poleceń

Podstawowa składnia wywołania programu prezentuje się następująco:

```
tsp-generator.exe --start <wartość> [opcje]
```

4.1.1 Dostępne argumenty i flagi

Program wymaga podania rozmiaru startowego oraz przyjmuje szereg argumentów opcjonalnych modyfikujących proces generowania:

- **--start <wartość>** (argument wymagany) – określa początkową liczbę wierzchołków dla pierwszej instancji w generowanej serii.
- **--step <wartość>** (argument opcjonalny, domyślnie: 1) – rozmiar kroku, o jaki zwiększana jest liczba wierzchołków w każdej kolejnej generowanej instancji.
- **-c, --count <wartość>** (argument opcjonalny, domyślnie: 1) – łączna liczba instancji (plików) do wygenerowania w ramach jednego uruchomienia.
- **-s, --size <wartość>** (argument opcjonalny, domyślnie: 1000) – maksymalna wartość współrzędnych (na osiach X i Y) dla losowanych wierzchołków.
- **-d, --dir <ścieżka>** (argument opcjonalny, domyślnie: **data**) – ścieżka do katalogu docelowego, w którym zostaną zapisane pliki z rozszerzeniem **.txt**.
- **--help** – wyświetla na standardowym wyjściu instrukcję obsługi programu.

4.2 Przykłady użycia

Uruchomienie podstawowe (pojedyncza instancja)

```
tsp-generator.exe --start 50
```

Program wygeneruje pojedynczy graf składający się z 50 wierzchołków, wylosuje współrzędne z przedziału od 0 do 1000 i zapisze wynik w domyślnym katalogu **data**.

Generowanie serii instancji testowych

```
tsp-generator.exe --start 100 --step 10 -c 10 -d test_set
```

Polecenie wygeneruje serię 10 plików (**-c 10**), zaczynając od rozmiaru 100 wierzchołków (**--start 100**) i zwiększając go o 10 w każdym kolejnym kroku (**--step 10**). Pliki zostaną zapisane w katalogu **test_set** (**-d**).

Generowanie na mniejszej płaszczyźnie współrzędnych

```
tsp-generator.exe --start 20 --size 500
```

Program wygeneruje graf 20-wierzchołkowy, w którym maksymalna wartość współrzędnych punktów na osiach X i Y nie przekroczy 500 (**-size**).

5 Instrukcja uruchomienia programu

Aplikacja została zrealizowana w środowisku .NET 9 jako program konsolowy.

5.1 Składnia poleceń

Podstawowa składnia wywołania programu (zakładając, że skompilowany plik wykonywalny nosi nazwę `tsp-approximation.exe`) prezentuje się następująco:

```
tsp-approximation.exe <FilePath> [opcje]
```

5.2 Dostępne argumenty i flagi

Program przyjmuje jeden argument pozycyjny oraz dodatkowe flagi modyfikujące jego zachowanie:

- `<FilePath>` (argument wymagany) – argument pozycyjny podawany bezpośrednio po nazwie programu. Określa ścieżkę do pliku tekstowego zawierającego macierz sąsiedztwa grafu.
- `-o, --output <ścieżka>` (argument opcjonalny) – pozwala na zdefiniowanie ścieżki do pliku wyjściowego, w którym zapisany zostanie wynik działania wybranego algorytmu. Jeśli parametr ten zostanie pominięty, wynik zostanie wypisany wyłącznie na standardowe wyjście (konsolę).
- `-d, --double-tree` (flaga opcjonalna) – jej podanie wymusza użycie algorytmu 2-aproksymacyjnego. Jeśli flaga nie zostanie ustawiona, program domyślnie wykorzysta algorytm Christofidesa.
- `--help` – wbudowana flaga, która wyświetla na ekranie krótką instrukcję obsługi i opisy dostępnych argumentów w języku angielskim.

5.3 Przykłady użycia

Uruchomienie domyślne (algorytm Christofidesa):

```
tsp-approximation.exe graph.txt
```

Program wczyta graf z pliku `graph.txt`, wykona obliczenia za pomocą algorytmu Christofidesa, a wynik zaprezentuje bezpośrednio w konsoli.

Uruchomienie algorytmu podwójnego drzewa z zapisem do pliku:

```
tsp-approximation.exe graph.txt -d -o result.txt
```

Program wykorzysta algorytm 2-aproksymacyjny (`-d`) i zapisze sformatowany cykl Hamiltona oraz jego koszt do pliku `result.txt` (`-o`).

Alternatywny zapis z pełnymi nazwami flag:

```
tsp-approximation.exe graph.txt --double-tree --output result.txt
```

Wywołanie to jest w pełni równoważne z przykładem drugim, ale wykorzystuje długie aliasy poleceń.

Bibliografia

- [1] Nicos Christofides. „Worst-Case Analysis of a New Heuristic for the Travelling Salesman Problem”. W: *Operations Research Forum* 3.1 (mar. 2022), s. 20. ISSN: 2662-2556. DOI: [10.1007/s43069-021-00101-z](https://doi.org/10.1007/s43069-021-00101-z). URL: <https://doi.org/10.1007/s43069-021-00101-z>.
- [2] Thomas H. Cormen i in. *Wprowadzenie do algorytmów*. Warszawa: Wydawnictwo Naukowe PWN, 2012. Rozd. 35.2. ISBN: 9788301169114.
- [3] A.M.H. Gerards. „Chapter 3 Matching”. W: *Network Models*. T. 7. Handbooks in Operations Research and Management Science. Elsevier, 1995, s. 135–224. DOI: [https://doi.org/10.1016/S0927-0507\(05\)80120-3](https://doi.org/10.1016/S0927-0507(05)80120-3). URL: <https://www.sciencedirect.com/science/article/pii/S0927050705801203>.
- [4] Dilson Lucas Pereira. *Minimum-Cost-Perfect-Matching: Algorithms for Maximum Cardinality Matching and Minimum Cost Perfect Matching Problems in General Graphs*. <https://github.com/dilsonpereira/Minimum-Cost-Perfect-Matching>. 2011.